CS 5200 - Database Management Systems

Project Report

GreenTrack: Microgreens Order Management System

Group Name
HsuCCardenasM

Group Members
Chia-hsiang Hsu Tai
Mishell Cardenas Espinosa

Link to Video Report: cs5200_demo.mp4 *(alternative link is provided in Appendix)*

README

The software used in this program can be listed below. To download the software please follow the instructions in the links provided below each software.

- *VS Code*
  - Link to download VSCode can be found below:
  - https://code.visualstudio.com/download
- *Python*
  - Link to download Python can be found below:
  - https://www.python.org/downloads/
- *MySQL*
  - Link to download MySQL can be found below:
  - https://dev.mysql.com/downloads/installer/
- *MySQL Workbench*
  - Link to download MySQL Workbench can be found below:
  - https://dev.mysql.com/downloads/workbench/
- *Node.js*
  - Instructions to download Node.js can be found below:
  - https://nodejs.org/en/download/current

This project was developed and tested on MacOS.

1. Open the zip file
2. Open the project folder in VSCode.
3. Open and run the provided MySQL database dump (*microgreen_db_dump.sql*) in MySQL Workbench. You can find it in the folder "db".
4. Go to the "api" folder, then go to the source folder and you should see a *.env* file. Open the *.env* file and add your username and password to the *.env* file and make sure to save:

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=*add username*
DB_PASSWORD=*add password*
DB_NAME=microgreens_db
```

5. Open the terminal and ensure your terminal is opened at the root of the project directory.
6. Split your VSCode terminal into two panes.
7. In the first terminal, navigate to the "api" folder:

```
cd api
```

8. Create a virtual environment (macOS/Linux):

```
python -m venv .venv
```

9. Activate the virtual environment (macOS/Linux):

```
. ./.venv/bin/activate
```

10. Install the required dependencies for the backend:

```
pip install -r requirements.txt
```

11. Start the development server:

```
uvicorn src.main:app --reload --port 8000
```

12. In the second terminal, navigate to the UI folder:

```
cd ui
```

13. Install all Node.js and React dependencies:

```
npm install
```

14. Start the development server:

```
npm run dev
```

15. The project application should be running in 'http://localhost:5173/'
16. You will be prompted to log in. Currently, the project includes only the Admin view. As an admin user, you have full access to all features of the application.
    - **Username**: mishell@mb.com
    - **Password**: password

    Chrome may warn you that the password has been found in breaches. You can safely ignore this message.

17. To log out, click on the profile icon at the top right corner. Select Log Out.
18. If successfully, you should be redirected to the login page.

Technical Specifications

We created a Microgreen Management System built to support the operational workflows of Boston Microgreens LLC (BMG), a Boston-based vertical farm. This means that a lot of our design decisions were made based on current operations, to guarantee that this tool could be incorporated seamlessly. It's designed to manage different aspects of BMG's operating procedures, ranging from crop planning and germination tracking to order fulfillment, packaging, and delivery logistics. The application is built on a Python-based backend, a TypeScript and React frontend and a MySQL relational database.

We would like to thank Boston Microgreens for providing us with actual data for this project. Some sensitive information has been redacted or modified for this project, but the majority of the information has remained the same which allowed us to properly validate functionality of our application. Since dates are a very important aspect of the project (everything is based on crop or order timing), we have only added order for the months of December 2025 and January 2026. Outside those 2 months, there will be no data.

Backend: FastApi, Python, JWT (authentication)

The backend for this project is built using Python, using FastAPI as the framework. We decided to use FastAPI because it was a simple and fast framework to work with to build APIs. The backend is organized such that each domain (Crop, Client, Harvest, etc) is separated into its own router, which helps us separate concerns and keeps code more maintainable. Each router is exposed to their specific endpoints to interact or retrieve information from the database through stored procedures. We tried to avoid having SQL code in the backend.

To validate data coming from the API calls, we use Pydantic models. These help us make sure that all the data passing through the backend is of the correct format and the fields are of the correct type, which help prevent type errors later on in the frontend. We did our authentication using JWT (JSON Web Tokens), which we use to allow users to log into the application and have the backend properly validate their login credentials.

To connect the backend to our MySQL database, we used PyMySQL. It allows us to establish secure connections to the database to execute procedures and function efficiently. This helped us maintain a clear separation between the application backend and the database logic.

Frontend: Vite, TypeScript, and React

The frontend is built on React with TypeScript. The project uses Vite since it provides a development server and build tool. We didn't use a framework since we believed that Vite provided enough functionality for the scope of the project. The user interface is separated into pages, one per domain. Each page corresponds to a router in the backend, to make sure that functionality is separated in a structured way between frontend and backend.

Database: MySQL Relational Database

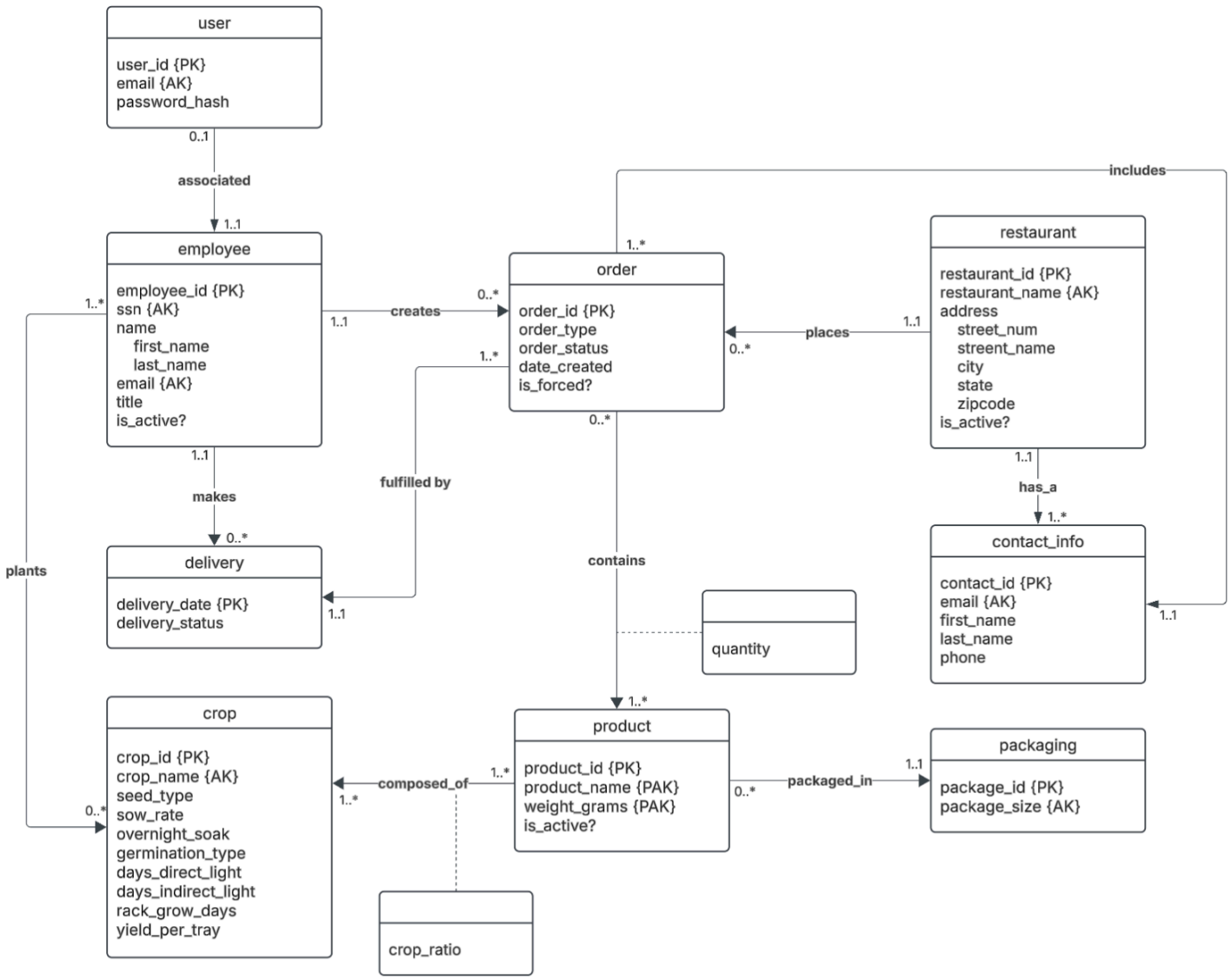
The application uses a MySQL relational database to store all the data. The database schema includes the tables "crop", "customer_order", "employee", "packaging", "product", "restaurant" and "users" among others. The schema uses primary keys, foreign keys, cascading updates, and Enums to enforce data consistency.

The backend interacts with the database through stored procedures. This makes it easier to manage database logic without having to make any modifications in the backend. We also use some functions, the database identifies information through unique ids, but in the frontend, we work with actual names. For example, for Amaranth the crop id is 1, so the database would use its crop id, but the frontend would display it as Amaranth, therefore our function maps the crop name to its corresponding crop id.

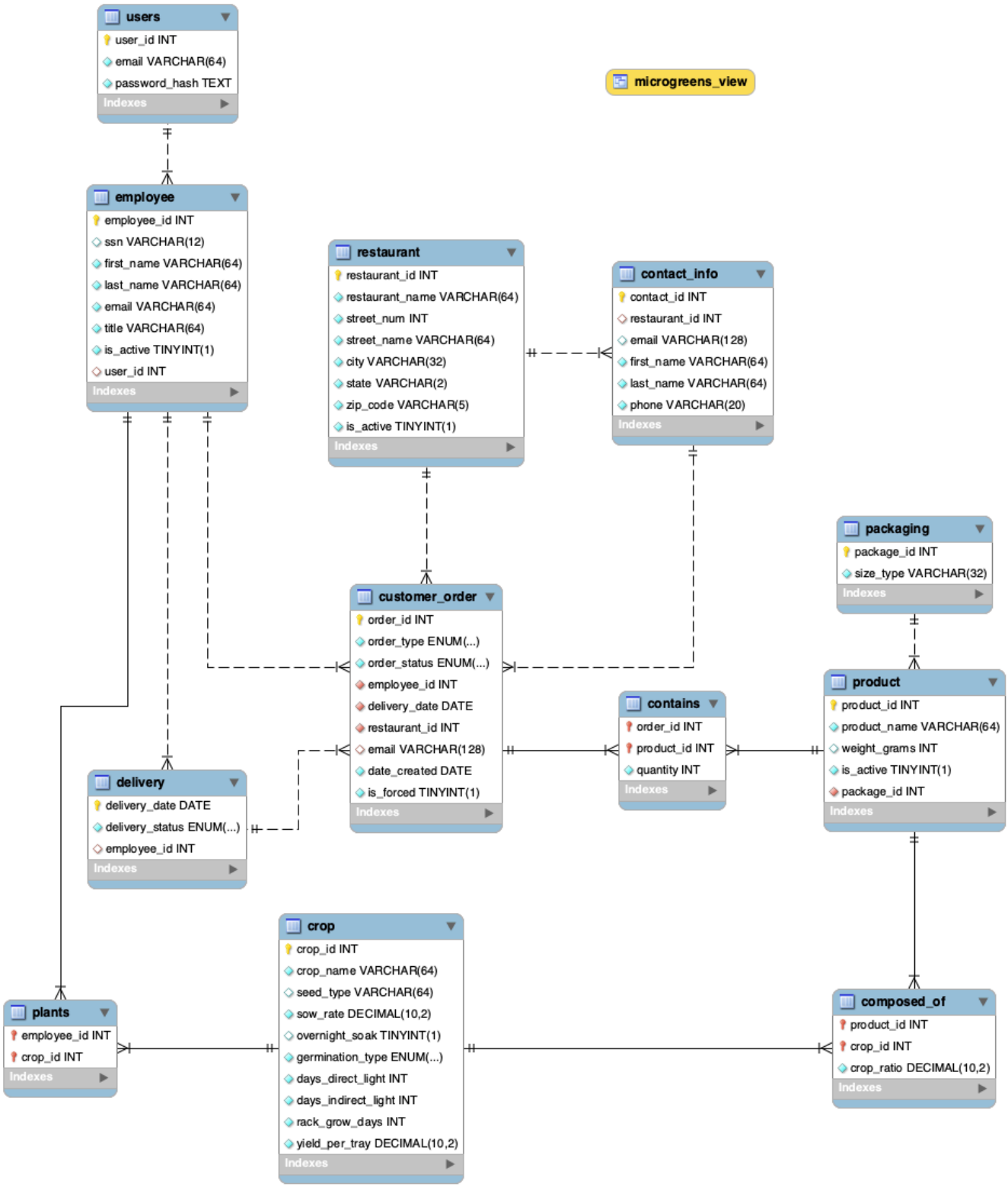
One of the most important aspects of the database is the View we created. This view joins together information that we need to calculate planting, germination, and grow rack timings. It is used extensively by our stored procedures since it stores most of the information regarding crops and orders. Using the view helps us reduce the amount of queries necessary to get all the information that we need, since it centralizes the data in one place.

Lastly, in our schema, we make most names unique even though names in general are not. This is because we are modeling our database based on how Boston Microgreens manages their data. For example, Boston Microgreens has a client called Davios, who has two locations in Boston, but they are tracked and known as Davios Seaport and Davios Arlington to uniquely identify them. Our database makes the same assumption where fields like client name or crop name are treated as unique.

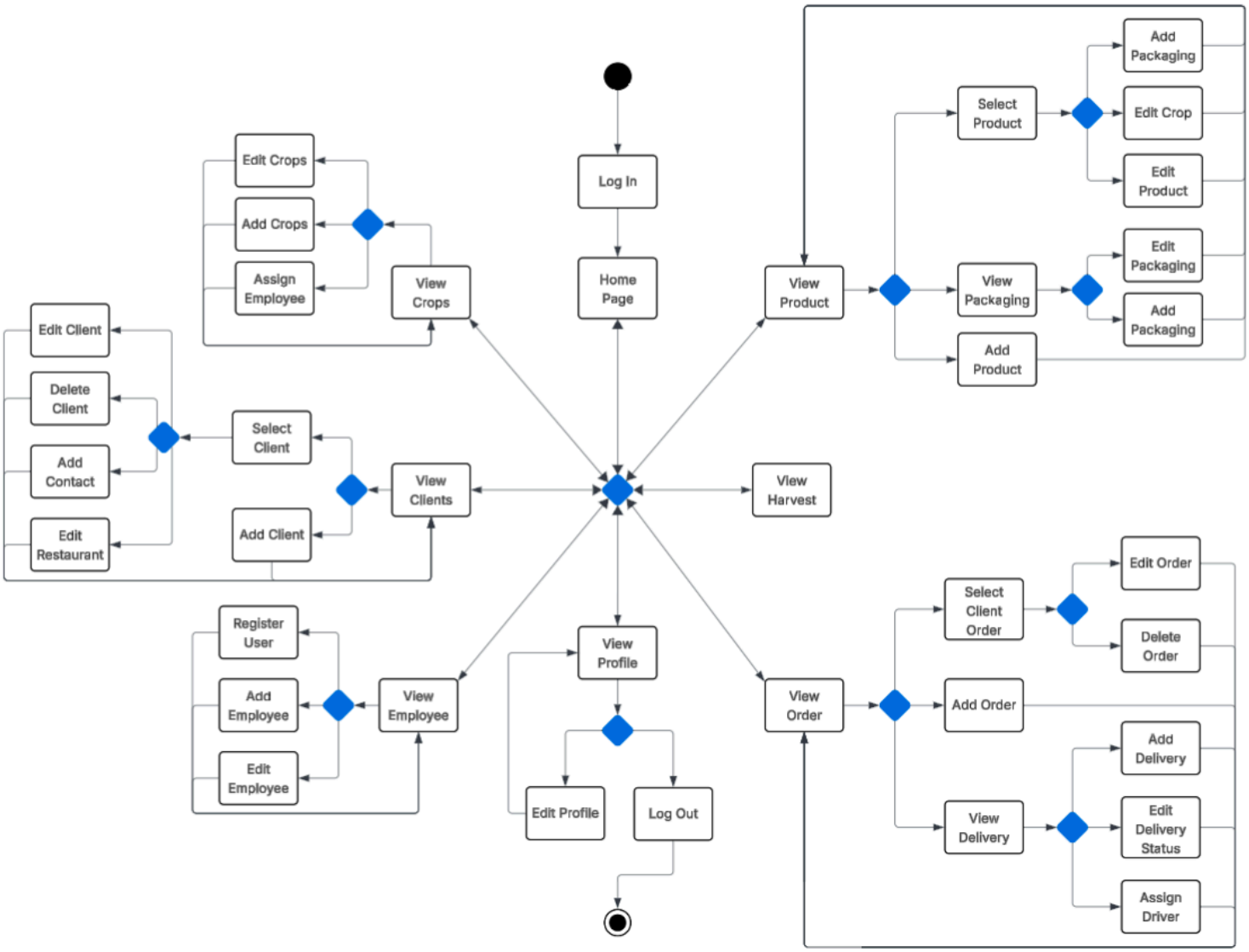
Conceptual Design



Logical Design



User Activity Description



Lesson's Learned

Technical Expertise Gained

For most of the members in our team, this was the first time where we built a fully functional web application where a database, backend, and frontend were created and connected. Therefore, this was a very enriching experience as it cemented the knowledge of how a full stack application is created from start to finish. For example, although we heard of endpoints before, this was our first time actually creating them and seeing how they help us connect the frontend and backend functionalities. Also, some of the programming languages used (such as React and Typescript) in the project were also fairly new to us, so this project provided a good opportunity to become more proficient in how to use them during the creation of an application.

Insights

Something that we realized as we started to connect the front end with the backend of our project was that our initial design of our database needed to change in order to implement the functionalities we wanted our application to have. Initially we developed a schema that we thought encompassed all the features the user required to handle the microgreens operations, but as we started to create the front end and visualize the user experience and interactions with the app, we noticed that we needed to reformat some our procedures as well as create new ones to handle actions that we didn't consider before. For example, in our initial schema design we didn't include a user entity which holds the information of all the users registered to use the app. We only thought of this when we thought about doing user authentication. Therefore, we had to update our schema and add procedures to be able to store and edit user information in our database such as their user name and passwords.

Additionally, we saw the importance of having consistency within our front end and backend operations. As we began to connect both sides, we had to constantly make sure that whatever was done by the user in the front end, also got reflected in the backend. This was important as the data needed to be consistent throughout, otherwise it could potentially affect the activities of the business in the case of a real world situation. This also ties to our realization of the importance of having a View in our project. The View helped us have the information that we needed all in one place, making it easier to access the data and offering overall versatility in how we wanted to use/present it.

In regards to time management, this project made us realize the importance of prioritizing certain tasks over others. At the very beginning, our team agreed to have a solid database so that our work with the front end could flow smoothly. This turned out to be a good decision as, although we ended up having to do some changes/updates down the line, we were able to build the main features without having any inconsistency in our data and database functionalities. This also ties to having good communication within our team, so that everyone is on the same page of the tasks

that need to be done and the deadlines that need to be met. Furthermore, our team had a nice dynamic in the sense that we helped each other out whenever someone got stuck with a bug, as well as asked questions and had discussions whenever someone didn't understand a technical concept.

Alternative Design and Approaches

Overall, our team is satisfied with how our application turned out. At the early stages of the project, we handrewed designs based on how we wanted the user interface to look like as well as what procedures we thought we would need to display and manipulate the data for each page. Having said this, we do believe that an alternative design could be applied for assigning employees to a task. Currently, an employee gets assigned a task based on the designated area (page) of where the task is. As an alternative, we thought about building another page that would only handle employee assignments to tasks. This way that operation is centralized for a better user experience and handling of data, instead of being scattered across the application. Furthermore, in the harvest page we would like that instead of displaying the harvesting information of a single day, the user could pick a date and the harvest information of the entire week gets displayed instead; this assimilates more to the actual operations of the business this app is built for. Lastly, we also consider having a better way of filtering the information of each page. For example in the Crop page, the user is only able to search by crop name. Having additional filters would make it easier for the user to manipulate the data and get the information that they need.

Work in Progress

An aspect of the project that isn't being handled is what happens when a token expires in the authentication part of the application. We noticed that when the token expires, it doesn't force a log out of the user. This can represent an issue because the user won't know when they are logged out, so any updates/changes done by the user will fail and not be registered by the application.

Additionally there are some places like in the Crop page, where when an input is provided we don't trim it to validate spaces. For example, if the user enters a product without a space and enters the same product with a space, our application treats them as different products instead of the same.

Future Work

Planned uses of the database

As we stated in our project proposal, the application that we built in this project was to help Boston Microgreens with their operation handling. Having finished the project satisfactorily, we plan to potentially pitch it to them so that they can use and integrate the application with their activities. As of right now, due to the scope of the project our database has a small portion of the data that Boston Microgreen has. Therefore, we plan to expand our database to hold any historical and new data that is currently not reflected in our database.

Potential areas for added functionality

As we mentioned in the alternative design and approaches section, we would like to improve the functionalities of data filtering, employee-to-task assignment, and the display of weekly harvesting information. Additionally, we would like to handle and solve the bug of logging out a user when the token expires; this would prevent inconsistencies when the user interacts with the application. Expanding on this idea, we would also like to make the user register button (which allows users to register a user so they can use the application) only available to the administrator, this way only they are able to register a new user and not other employees. Furthermore, add a restriction so that users are not able to edit each other's passwords which could cause confusion when accessing accounts. Lastly, if we expand our database to fit the data that Boston Microgreens has, this implies that we might need to change our procedures and other features to be able to handle this new information. One such update would be to, for example, include more information about the available crops, include pricing for products, handle inventory, and eventually provide business analysis features like expected revenue which will help the company have a clearer picture of their financial health.

Appendix

Alternative Link to Video Report:

https://drive.google.com/file/d/15nfLy-LJ8bV7xtVjn4qXwuO3GtiGYFyt/view?usp=drive_link

Link to Github Repository:

https://github.com/tchiahsu/microgreen_app